



Cours de Web dynamique : PHP–MySQL

Licence Mathématiques Appliquées

Septembre 2023



Formulaires HTML

Les objectifs des formulaires

- ❖ Un formulaire permet d'effectuer différentes choses :
 - ✓ Saisir du texte, des mots de passe
 - ✓ Choisir un élément dans une liste
 - ✓ Afficher un résultat
 - ✓ Lancer des actions (appel de script ou ouverture de pages) avec des boutons
 - ✓ Envoyer les informations saisies par mail

Les formulaires du HTML

- ❑ Toute la partie formulaire de la page doit se trouver entre les marqueurs **<FORM> ...</FORM>**
- ❑ Cette balise sert de conteneur pour accueillir les éléments ou composants du formulaire
 - Un bouton, une ligne de texte, une liste de choix...
- ❑ Chacun de ces composants s'y trouve déclaré et est créé individuellement par une balise qui commence généralement par **INPUT** ou par **SELECT**

Propriétés et méthodes de FORM

❖ Liste des propriétés :

- **name** : Nom du formulaire
- **action** : Adresse du script de serveur à exécuter
- **method** : Méthode d'appel du script (get ou post)
- **enctype** : Type d'encodage du formulaire
- **target** : Destination du retour du formulaire

❖ Liste des méthodes

- **submit** : Déclenche l'action du formulaire
- **reset** : Réinitialise les données du formulaire avec les valeurs par défaut

❖ Liste des événements

- **onSubmit** : Détecte la soumission du formulaire
- **onReset** : Détecte la réinitialisation

Saisie par ligne de texte

❖ Ligne de texte

```
<INPUT TYPE="text" NAME="Nom du champ"  
      VALUE="texte initial" SIZE=longueur ou  
      MAXLENGTH=nb >
```

❖ L'objet INPUT permet d'afficher :

- des champs texte
- des boutons
- des radio-boutons
- des cases à cocher
- un champ caché
- des champs password (***)
- les boutons spéciaux RESET et SUBMIT

Pas de blanc
entre < et **INPUT** !!

Propriétés de INPUT

❖ Liste complète des propriétés :

- **Name** : nom du champ
- **Type** : type du champ *text, button, radio, checkbox, submit, reset*
- **Value** : Libellé texte
- **Defaultvalue** : valeur par défaut du champ (utile avec reset)
- **Size** : taille du champ
- **Maxlength** : taille maximale du champ de type texte
- **Checked** : case à cocher ou radio bouton coché ou non
- **Disabled** : grisé (modification impossible par le visiteur)
- **readOnly** : lecture seule
- **Class** : nom de la classe de style
- **Style** : chaîne de caractères pour le style

Champ de saisie Bouton

❑ `<INPUT type="button" value="Détruire la session">`



❑ Propriétés acceptées :

name | value | defaultvalue | size | maxlength | disabled | readOnly |
class | style

❑ Méthodes acceptées:

focus | blur | click (donne, reprend le focus, clic)

❑ Événements acceptés:

onFocus | onBlur | onClick

- onFocus détecte la prise de focus
- onBlur détecte la perte de focus
- onClick détecte le clic utilisateur

Liste de sélection

- ❖ Les listes sont définies par la balise `<SELECT>`. Cette balise définit la zone de la liste. Les lignes de contenu de la liste sont alimentées par la balises `<OPTION>`.

Voici la syntaxe générale pour créer une liste contenant des lignes :

```
<SELECT name="mono" size=1>  
  <OPTION value="1">ligne 1</OPTION>  
  <OPTION value="2">ligne 2</OPTION>  
  <OPTION value="3">ligne 3</OPTION>  
  <OPTION value="4">ligne 4</OPTION>  
</SELECT>
```

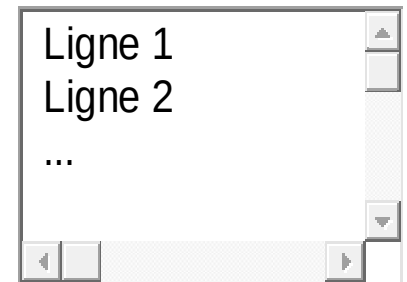


Formulaire : Zone de texte

❖ Zones de textes multi-lignes TEXTAREA

- ❖ L'objet TEXTAREA est essentiellement utilisé pour permettre au visiteur de saisir un texte assez long (message, descriptif...).
Voici la syntaxe classique pour définir une zone de texte :

- ❖ `<TEXTAREA name="texte" rows="5" cols="20">`Ligne 1
Ligne 2
...
`</TEXTAREA>`



Exemple de formulaire : unFormulaireComplet.html

Formulaires et PHP

- ❑ On associe un script PHP à un formulaire par :

```
<FORM action="ins.php" method="post">
```

... .

```
</FORM>
```

- ❑ Bouton déclenchant l'action du formulaire :

```
<INPUT type="submit" value="CLIC">
```

- ❑ Ce formulaire déclenche une action qui ici consiste à appeler le programme `ins.php`

- et à l'exécuter sur le serveur

- ❑ On va utiliser les noms définis pour les champs texte : `<INPUT type="text" name="nom">` pour récupérer les valeurs

Essai4.html

```
<FORM action="essai4.php" method="post">
```

NOM

```
<INPUT type=text name="nom"><BR>
```

PRENOM

```
<INPUT type=text name="prenom"><BR>
```

ADRESSE

```
<INPUT type=text name="adresse"><BR>
```

```
<INPUT type=submit value="CLIQUEZ ICI">
```

```
</FORM>
```

Essai4.html

Lorsqu'un formulaire est envoyé à un script PHP, toutes les variables du formulaire seront automatiquement disponibles dans le script sous forme de chaînes.

```
<html>
```

```
<body>
```

```
<H1 Mon quart d'heure de célébrité !
```

```
<BR>
```

```
  Bienvenue à Monsieur
```

```
<? php
```

```
print "<H1>" . $_POST['prenom'] ;
```

```
  print $_GET['nom'] . " </H1>" ;
```

```
?>
```

```
</body>
```

```
</html>
```

Fichier **essai4.php**

Noms des champs du
formulaire

Ex. de formulaire écrit en PHP

Ici on saisit le login et pwd, et on appelle une fonction de vérification effectuée dans `check.php`.

On observera les guillemets internes à la chaîne:

```
echo "<form method=\"post\" action=\"check.php\">  
Login : <input type=\"text\" name =\"login\"  
value=\"\$login\" /><br />  
Password : <input type=\"password\" name  
= \"pass\" value=\"\$pass\" /><br />  
<input type=\"submit\" value=\"Identifier\" />  
</form>";
```

Préférer les apostrophes aux guillemets doubles quand il n'y a pas besoin des deux, c'est plus rapidement interprété

Fonctions `trim()` et `empty()`

- ❑ La fonction `trim()` retire les espaces en début et fin de chaîne
 - cela permet de ne pas traiter des informations vides (un utilisateur pourrait très bien n'entrer que des espaces dans le formulaire)
- ❑ La fonction `empty()` permet de savoir si une variable (chaîne ou bouton) a une valeur ou non.
- ❑ Cf un exemple avec `formulaire.html` et `formul1.php` ci-après

Formulaire.html

```
<form action="formulaire.php" method="post">
  <table>
    <tr>
      <td>Nom :</td>
      <td><input type="text" name="nom" value=""></td>
    </tr>

    <tr>
      <td>Prénom :</td>
      <td><input type="text" name="prenom" value=""></td>
    </tr>

  </table>

  <input type="submit" value="envoyer" name="send">

</form>
```


Formulaire.php

```
<?php
$bouton = $_POST['send'];
if(!empty($bouton)) {
    $nom = trim($_POST['nom']);
    $prenom = trim($_POST['prenom']);
    if(!empty($nom) && !empty($prenom)) {
        echo 'Bonjour, ' . $prenom . ' ' . $nom;
    }
    else {
        echo "vous n\'avez pas rempli tous les champs";
    }
}
?>
```

Accéder à des variables de formulaire HTML

```
<?php
// Disponibles depuis PHP 4.1.0
echo $_POST['nom']; // pour les variables utilisateur
echo $_REQUEST['nom']; // pour les variables d'environnement

import_request_variables('p', 'p_'); // Importe les variables de
// POST dans l'environnement global, avec le préfixe 'p_' ->
// $_POST['nom'] ⇔ $p_nom
echo $p_username;
// Disponibles depuis PHP 3:
echo $HTTP_POST_VARS['username'];

// echo $username;
// serait disponible si la directive 'register_globals = on' dans
php.ini.
// Depuis PHP 4.2.0 la valeur par défaut de cette directive est à
'off'
// Utiliser ou présumer sur cette méthode est
// fortement déconseillé; c'est une sécurité
// importante.
?>
```

Accès aux variables avec GET

- ❑ Utiliser un formulaire de type GET est similaire, hormis le fait qu'il faut utiliser les variables pré-définies de GET à la place.
- ❑ GET s'applique aussi à la *QUERY_STRING* (les informations disponibles après le '?' dans une URL).
 - De ce fait, par exemple,
http://www.example.com/test.php?id=3 contient les données de GET, qui sont accessibles via `$_GET['id']`

Passer des variables par l'adresse URL avec `_GET`

`http://www.toto.fr/index.php?var=12&a=1&b=2`

Dans la page `www.toto.fr/index.php`, les 3 variables sont utilisables :

```
$_GET['var']=12
```

```
$_GET['a']=1
```

```
$_GET['b']=2
```

*Pas de guillemet pour les chaînes, espaces autorisés, valeurs numériques bien récupérées. Mais attention avec les caractères spéciaux: utiliser **urlencode**, **urldecode**,*

`http://www.cdprof.com/formation/formation_php1b.php`

Variables de
type String:

```
http://www.toto.fr/index.php?nom=cow&prenom=flying
<?php
$nom = $_GET['nom'];
$prenom = $_GET['prenom'];
echo 'Bonjour, ' . $prenom . ' ' . $nom;
?>
```

Convention d'écriture PHP

- ❑ Localiser les parties "programme" en début de fichier
 - ✓ avant tout code HTML
- ❑ Cela permet de bien séparer la partie **traitement**, maintenue par le programmeur
 - ✓ de **l'affichage**, dévolue au Web designer